The Journal of Open Source Software

xr_fresh: Automated Time Series Feature Extraction for Remote Sensing & Gridded Data

Michael L. Mann [1]

1 The George Washington University, Department of Geography & Environment, Washington DC 20052

Abstract

xr_fresh is a Python library for automated feature extraction from gridded time series data, such as satellite imagery, climate model outputs, and sensor arrays. Building on the methodology of tsfresh, xr_fresh extends this approach to pixel-level temporal sequences common in observational data such as from earth observation or repeat photography data. It computes a comprehensive set of statistical, trend, and distribution-based features for each pixel, enabling scalable preprocessing for classical machine learning. The library is optimized for large-scale applications through parallelized computation using xarray, Dask, Ray, and JAX. It also includes advanced interpolation techniques for handling missing data and GPU-accelerated kernel PCA for dimensionality reduction.

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 21 May 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Statement of need

Gridded time series data from satellites, climate models, camera feeds, and sensors contain rich temporal information for applications like crop type classification and yields, anomaly detection, robotics, quality control, environmental monitoring, and natural resource management (Delince et al., 2017; Hufkens et al., 2019; Michael L. Mann et al., 2019; Michael L. Mann & Warner, 2017; Mumuni & Mumuni, 2024). Efficiently extracting relevant time series features at scale remains challenging, necessitating automation (Faouzi, 2022; Li et al., 2020). Inspired by `tsfresh`, we introduce `xr_fresh`, tailored specifically for gridded time series by automating the extraction of time series features on a pixel-by-pixel basis (Christ et al., 2018).

Currently, there is no method to rapidly extract a comprehensive set of features from gridded time series data, such as those derived from remote sensing imagery. Existing packages like `tsfresh` are not optimized for the unique structure of gridded time series data. This limitation hinders the ability to efficiently analyze and model these datasets, particularly in the context of remote sensing applications where large volumes of data are generated.

To address this gap, `xr_fresh` automates the extraction of salient temporal and statistical features from each pixel time series. Using automated feature extraction, `xr_fresh` reduces manual intervention and improves reproducibility in remote sensing workflows.

Problems and Background

An image time series can be represented as a three-dimensional array with spatial dimensions x and y, and temporal dimension z. Each pixel at location (x_i, y_j) holds a time series:

$$\mathcal{D} = \{X_{i,j} \in \mathbb{R}^T \mid i = 1, \dots, H; j = 1, \dots, W\}$$

where H and W are the height and width of the image, and T is the number of temporal observations (e.g. monthly composites or daily acquisitions).

To prepare these data for use in supervised or unsupervised machine learning, each pixel time series $X_{i,j} = (x_{i,j,1}, x_{i,j,2}, \dots, x_{i,j,T})$ is transformed into a feature vector:

$$\vec{x}_{i,j} = \left(f_1(X_{i,j}), f_2(X_{i,j}), \dots, f_M(X_{i,j})\right)$$

where each f_m is a time series feature extraction function (e.g. mean, variance, trend, autocorrelation), and M is the total number of extracted features.

A visual representation of this transformation is shown in Figure 1.

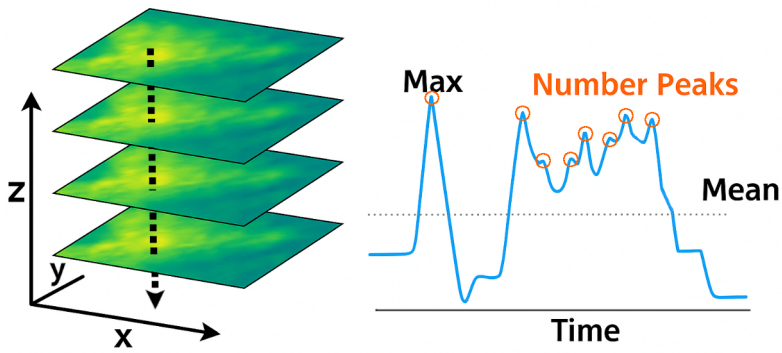

Figure 1: Feature Extraction Process

This results in a 2D design matrix of features for the entire image:

$$\mathbf{X}_{\text{features}} \in \mathbb{R}^{H \times W \times M}$$

This transformation effectively reduces the temporal complexity while preserving informative temporal patterns, enabling efficient training of models or aggregation to coarser units (e.g., fields or regions).

Additional static features (e.g., soil type, elevation), can be concatenated:

$$\vec{x}_{i,j}^{\text{final}} = \left[\vec{x}_{i,j} \,|\, \vec{a}_{i,j}\right] \in \mathbb{R}^{M+U}$$

where $\vec{a}_{i,j} \in \mathbb{R}^U$ represents the U univariate attributes at pixel (i, j).

Time Series Feature Set

The documentation summarizes the suite of time series features extracted by the `xr_fresh` module from gridded data. These features are designed to characterize the temporal behavior of each pixel (x_i, y_j). By including a diverse set of statistical, trend and distribution-based metrics, `xr_fresh` enables a detailed and scalable analysis of temporal patterns (Jin et al., 2022; Venkatachalam et al., 2024). Additional features can be added to the library as needed, and users can also define custom feature extraction functions.

Interpolation

The `xr_fresh` library includes functionality to interpolate missing values pixel-wise in gridded data. The interpolation methods implemented in `xr_fresh` are designed to be computationally efficient and can handle large datasets effectively. The module supports advanced interpolation techniques including linear, nearest-neighbor, cubic, and univariate spline interpolation (Virtanen et al., 2020).

Formally, for a fixed pixel (i, j), let the time series be:

DRAFT

$$X_{i,j} = (x_{i,j,1}, x_{i,j,2}, \dots, x_{i,j,T})$$

where some $x_{i,j,t}$ may be missing due to clouds or sensor gaps. The interpolation estimates these missing values by fitting a function $f(t)$ to the observed time steps $\{t_k \in [1, T] \mid x_{i,j,t_k}$ is observed$\}$. The interpolated value at time t is:

$$\hat{x}_{i,j,t} = f(t), \quad \text{for } x_{i,j,t} \text{ missing}$$

The function $f(t)$ may take the form of: 1) linear interpolation, 2) nearest neighbor, 3) cubic spline interpolation, or 4) univariate spline interpolation. If acquisition times are irregular, the time t is replaced by a datetime indexes.

Dimensionality Reduction

For high-dimensional inputs or when the number of bands/time steps is large, dimensionality reduction can improve model performance. `xr_fresh` integrates a GPU/CPU-parallelized Kernel Principal Component Analysis (KPCA) module (Pedregosa et al., 2011). The KPCA implementation samples valid observations for training, fits the kernel model, and projects each pixel's time series into a lower-dimensional space.

Software Framework

`xr_fresh` achieves scalability by employing a combination of parallel and distributed computing strategies. During feature extraction, functions are applied in parallel across spatial windows with Dask and xarray, which provide lazy evaluation, chunked computation (Graesser & Mann, 2025; Hoyer & Hamman, 2017; Rocklin, 2015). Seamlessly integrated into the parallel pipeline and can leverage accelerated libraries like JAX, NumPy, ray, numba or PyTorch for additional speedup [Bradbury et al. (2018);lam2015numba;Harris et al. (2020);Moritz et al. (2018)]. Together, these strategies ensure that methods are highly scalable, for use on large-scale datasets.

Example: Precipitation In Africa

We apply `xr_fresh` methods to a dataset of monthly precipitation estimates in Africa (Figure 2) (Funk et al., 2015). The goal is to extract features from the time series data, enabling subsequent analysis and modeling. The `extract_features_series` function takes a list of files, a dictionary of desired features.

DRAFT

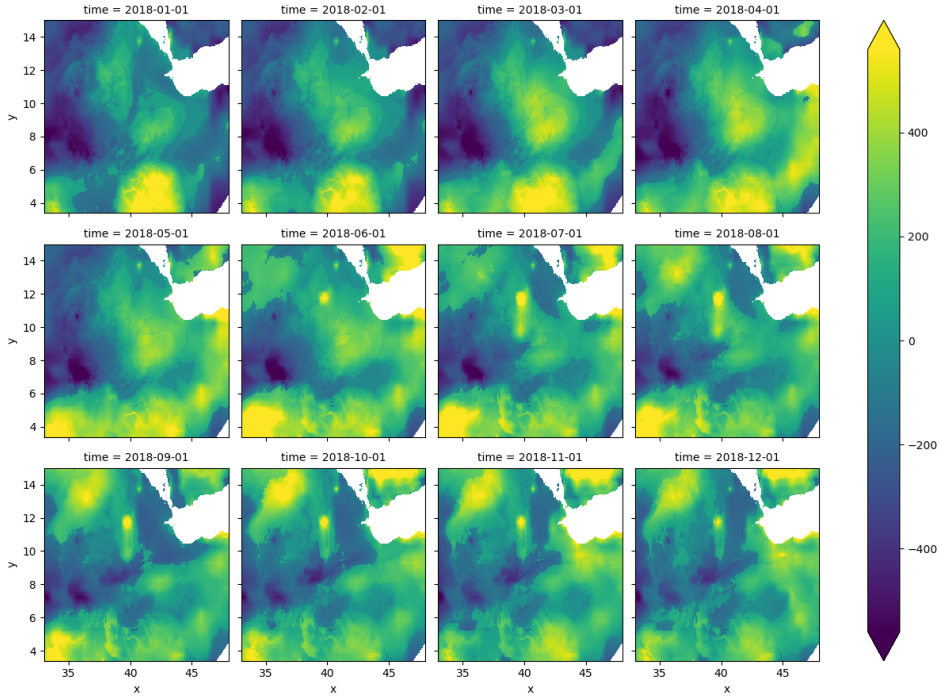

Figure 2: Precipitation input data

```
# create list of desired series and arguments
feature_list = {
    "minimum": [{}],
    "abs_energy": [{}],
    "doy_of_maximum": [{"dates": dates}],
    "mean_abs_change": [{}],
    "ratio_beyond_r_sigma": [{"r": 1}, {"r": 2}],
    "symmetry_looking": [{}],
    "sum": [{}],
    "quantile": [{"q": 0.05}, {"q": 0.95}],
}
from xr_fresh.extractors_series import extract_features_series

# Extract features from the geospatial time series
extract_features_series(image_list, feature_list, band_name, out_dir,
                        num_workers=12, nodata=-9999)
```

DRAFT

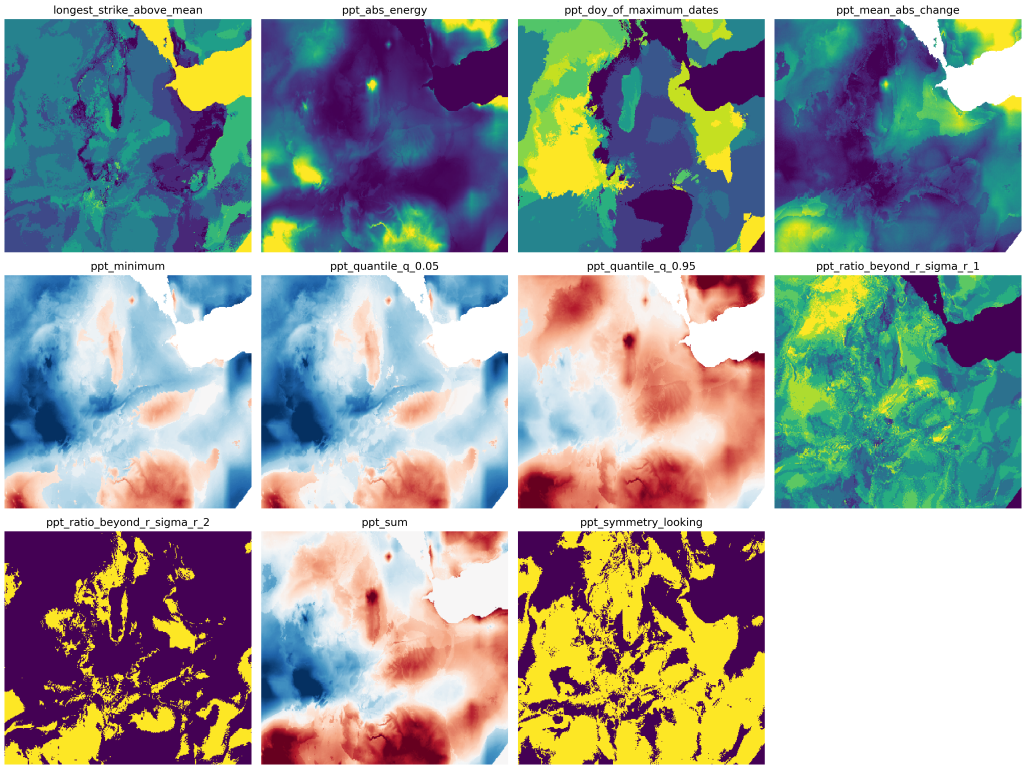

Figure 3: Time series feature set

The extracted features found in Figure 3 can then be used in a variety of applications.

Conclusions

`xr_fresh` is a powerful and efficient tool for automated feature extraction from gridded time series. Using advanced statistical methods and parallel computing, it enables the extraction of a comprehensive set of features that can significantly enhance the performance of machine learning models. Integration with existing Python geospatial libraries ensures that `xr_fresh` is easy to use and can be seamlessly incorporated into existing machine learning workflows. It also provides advanced interpolation and dimensionality reduction capabilities, addressing common challenges in remote sensing data analysis.

DRAFT

References

Bradbury, J., Frostig, R., Hawkins, P., Johnson, M. J., Leary, C., Maclaurin, D., Necula, G., Paszke, A., VanderPlas, J., Wanderman-Milne, S., & Zhang, Q. (2018). *JAX: Composable transformations of Python+NumPy programs* (Version 0.3.13). http://github.com/jax-ml/jax

Christ, M., Braun, N., Neuffer, J., & Kempa-Liehr, A. W. (2018). Time series FeatuRe extraction on basis of scalable hypothesis tests (tsfresh – a python package). *Neurocomputing*, *307*, 72–77. https://doi.org/https://doi.org/10.1016/j.neucom.2018.03.067

Delince, J., Lemoine, G., Defourny, P., Gallego, J., Davidson, A., Ray, S., Rojas, O., Latham, J., & Achard, F. (2017). Handbook on remote sensing for agricultural statistics. *GSARS: Rome, Italy*. https://doi.org/10.13140/RG.2.2.13259.69920

Faouzi, J. (2022). Time series classification: A review of algorithms and implementations. *Machine Learning (Emerging Trends and Applications)*. https://doi.org/10.5772/intechopen.1004810

Funk, C., Peterson, P., Landsfeld, M., Pedreros, D., Verdin, J., Shukla, S., Husak, G., Rowland, J., Harrison, L., Hoell, A., & others. (2015). The climate hazards infrared precipitation with stations—a new environmental record for monitoring extremes. *Scientific Data*, *2*(1), 1–21. https://doi.org/10.1038/sdata.2015.66

Graesser, J., & Mann, M. (2025). *GeoWombat (v2.1.22): Utilities for geospatial data*. Zenodo. https://doi.org/10.5281/zenodo.15483823

Harris, C. R., Millman, K. J., Walt, S. J. van der, Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., Kerkwijk, M. H. van, Brett, M., Haldane, A., Río, J. F. del, Wiebe, M., Peterson, P., … Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, *585*, 357–362. https://doi.org/10.1038/s41586-020-2649-2

Hoyer, S., & Hamman, J. (2017). Xarray: N-D labeled arrays and datasets in Python. *J. Open Res. Software*. https://doi.org/10.5334/jors.148

Hufkens, K., Melaas, E. K., Mann, M. L., Foster, T., Ceballos, F., Robles, M., & Kramer, B. (2019). Monitoring crop phenology using a smartphone based near-surface remote sensing approach. *Agricultural and Forest Meteorology*, *265*, 327–337. https://doi.org/10.1016/j.agrformet.2018.11.002

Jin, G., Li, F., Zhang, J., Wang, M., & Huang, J. (2022). Automated dilated spatio-temporal synchronous graph modeling for traffic prediction. *IEEE Transactions on Intelligent Transportation Systems*, *24*(8), 8820–8830. https://doi.org/10.1109/TITS.2022.3195232

Li, X., Kang, Y., & Li, F. (2020). Forecasting with time series imaging. *Expert Systems with Applications*, *160*, 113680. https://doi.org/10.1016/j.eswa.2020.113680

Mann, Michael L., & Warner, J. M. (2017). Ethiopian wheat yield and yield gap estimation: A spatially explicit small area integrated data approach. *Field Crops Research*, *201*, 60–74. https://doi.org/https://doi.org/10.1016/j.fcr.2016.10.014

Mann, Michael L., Warner, J. M., & Malik, A. S. (2019). Predicting high-magnitude, low-frequency crop losses using machine learning: An application to cereal crops in ethiopia. *Climatic Change*, *154*(1), 211–227. https://doi.org/10.1007/s10584-01902432-7

Moritz, P., Nishihara, R., Wang, S., Tumanov, A., Liaw, R., Liang, E., Elibol, M., Yang, Z., Paul, W., Jordan, M. I., & Stoica, I. (2018). Ray: A distributed framework for emerging AI applications. *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, 561–577. https://doi.org/10.48550/arXiv.1712.05889

Mumuni, A., & Mumuni, F. (2024). Automated data processing and feature engineering for

DRAFT

deep learning and big data applications: A survey. *Journal of Information and Intelligence*. https://doi.org/10.1016/j.jiixd.2024.01.002

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, *12*, 2825–2830. https://doi.org/10.5555/1953048.2078195

Rocklin, M. (2015). Dask: Parallel computation with blocked algorithms and task scheduling. *Proceedings of the 14th Python in Science Conference*, 130–136. https://doi.org/10.25080/Majora-7b98e3ed-013

Venkatachalam, S., Kacha, D., Sheth, D., Mann, M., & Jafari, A. (2024). *Temporal patterns and pixel precision: Satellite-based crop classification using deep learning and machine learning*. George Washington University, Department of Geography & Environment; Data Science Program.

Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., & others. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in python. *Nature Methods*, *17*(3), 261–272. https://doi.org/10.1038/s41592-019-0686-2

DRAFT